Learning Apples to Apples Using Large Language Models and Q-Learning

Anonymous Submission

Abstract

This paper focuses on the development of two artificial intelligence agents designed to rationally play Apples to Apples, a card game that requires subjective decision-making. One agent (labelled agent A) uses a combination of word association and clustering, and another (labelled agent B) uses a reinforcement learning policy to grow towards making subjectively right decisions throughout the game. Using word association and vectorization, Agent A can evaluate card meanings, while additional clustering techniques allow it to group and predict similar judgments. Reinforcement learning allows Agent B to follow a policy, hoping to improve its effectiveness in gameplay through learned experience. The results indicate that not only can both agents effectively participate in the game, but the reinforcement learning agent is also capable of making decisions that reflect a developing understanding of the state space. This paper hopes to highlight the potential for artificial intelligence to handle subjective environments contributing to advancements of agents in socially dominated games.

Introduction

The objective of this research is to design an artificial intelligence agent capable of rationally playing the card game Apples to Apples. This task requires not only the ability to make strategic decisions when playing cards but also the ability to make social judgments. For the agent to succeed in rationally playing Apples to Apples it must be able to not only play the game as a player but also as a judge.

Apples to Apples is a board game introduced by Mattel in 1999, in which the goal is for players to accrue enough points to win the game through having their card chosen[1]. Each round, one of the players acts as the judge and draws a "green card" which contains an adjective. The remaining players anonymously submit one "red card" from their hand which contains a noun. The judge then picks the red card submitted that they feel best matches the green card. The player whose card is chosen by the judge is considered the winner for the round and gains one point. The number of cards in a player's hand and number of points required to win change depending on the number of players playing the game. Once a player accrues enough points from winning rounds, they win the game.

The goal of this project is to introduce a rational artificial intelligence agent into more of a socially dominated environment, instead of traditional rule-based environments This paper outlines the methods used to create artificial intelligence agents A and B, including word association, clustering, and reinforcement learning. This project hopes to creatively contribute to developing artificial intelligence agents in social environments, such as the one presented by Apples to Apples.

Related Work

Related work on AI agents for word-based games shows that reinforcement learning struggles with big state spaces and natural language processing. In the paper "A survey of text games for reinforcement learning informed by natural language" (Osborne 2022), the agent's state space relies on a "reward signal defined on the game's win condition"(Osborne 2022) to decide its next move. To make the state space even more reduced, similar text was replaced with symbolic tokens, for example, "Dwayne Johnson" and "Zac Efron" could be grouped under the symbolic token of ACTOR. Similarly, work done by Narasimhan, Kulkarni, and Barzilay (2015) shows that use of deep reinforcement learning can help capture the semantics of different game states (Narasimhan; Kulkarni; Barzilay 2015).

Methodology

Word Embeddings

The Apples to Apples agent was designed to perform three main tasks; the ability to play, judge, and adapt. Agent A's decision-making process was built on natural language processing using the SpaCy[2] library in Python which provides pre-trained word embeddings that Agent A uses to calculate word association vectors for both red and green game cards.

[1] Apples to Apples manual used can be found here: https://service.mattel.com/instruction_sheets/N1488-0920.pdf

[2] SpaCy library can be found here: https://spacy.io/

When acting as a player, Agent A compares the word vectors for its red cards against the current green card in play and selects the closest cosine association values determined by finding the minimum distance between an available red card word vector and the green card in play. When assuming the role of the judge, Agent A evaluates red cards submitted by other players or agents, similarly, choosing the red card with the minimum cosine distance, or closest similarity to the green card. While Agent A could certainly play the game, there was no notion of adaptation or learning, only playing based on mathematically formulated notions of what red card is most similar. This led to the development of the reinforcement learning agent, Agent B.

Reinforcement Learning

Reinforcement learning agents have had significant success with problems revolving around the play of games (Szita 2012) (Crespo, Wichert 2020) (Ghory 2004) (Leslie, Collins 2005) (Kim, Kim 2019) (Wang, Yang 2020). This made choosing to use a reinforcement learning agent an obvious choice for this project. The choice to move forward with the use of the Q-learning algorithm was made due to the algorithm's popularity with games (Leslie, Collins 2005) (Kim, Kim 2019) (Wang, Yang 2020).

Apples to Apples is a multiplayer game where multiple players select from their hand a red card containing a noun that they believe matches in some way the green card containing an adjective. The game is formulated in a way that no player can tell what each other player played that turn, and no player can see what another player plays before they play their own red card. This was considered during the formulation of the state space for Agent B; the agent only looks at its own hand of red cards and the green card currently in play. Thus, each state is represented by the green card in play and Agent B's hand of red cards. The state-action pairs are then represented by the state, followed by each red card in Agent B's hand as an action. Technically, the state-action pair value function is represented by a dictionary containing each green card as a key, each associated with a dictionary using all the possible unique hands as keys, and then associated with an array containing the values of playing each card. Hands are sorted before Agent B sees them so that the ordering is consistent with regards to key search and value lookup. In short, the state-action value function for Agent B is given as a dictionary of dictionaries of arrays.

Apples to Apples nicely separates into episodes in one of two ways: either in terms of a full game or in terms of individual rounds. Agent B's goal for this problem is to accrue enough points to win the game, so it could make sense for one to reward the agent every time it wins a game. This however does not make much sense for Agent B's state representation of the environment; since Agent B does not naturally distinguish between the rounds of a game, the only round that will update Agent B's value function is the final round. This would result in significantly slower learning, an inability to learn how to effectively play Apples to Apples, or both. The choice was then made to reward Agent B for each round won as opposed to each game won. This should effectively achieve the same goal, as to maximize Agent B's reward, it needs to win enough rounds to win the game.

The reward approach for the Q-learning Agent B was giving a penalty for losing (-0.5 reward on loss of round) and a reward of +1 for winning the round. The thought behind penalizing the agent for losing rounds was to encourage exploration; if Agent B loses a round and is not penalized, then the update step of Q-learning does not update the value for choosing that card in that state if the value of choosing that card is currently zero. If a small penalty is applied for losing a round, then Agent B will not choose that card in that state unless it has at least tried every other card (and subsequently lose the round with those cards). Agent B used a learning rate of 0.5 and discount rate 1.

Clustering

The number of cards in the game proved issues in the amount of state space Agent B had to learn which resulted in the learning being extremely inefficient. Data preprocessing techniques such as clustering was performed to reduce the amount of data we were working with. Sentence embeddings of all the red and green cards were first normalized and then clustered using the KMeans algorithm. This allowed for all the cards to be grouped together into clusters resulting in a significant reduction of our data. Initial trials of 100 games were first performed on 10, 25, 50, and 100 numbers of clusters and the average sum of reward was looked at. From these results, a local maximum was determined at the 50 clusters then further investigation was performed over 20 games using 40, 45, 55, 60, 65, 70, and 75 numbers of clusters. Over these 20 games, 65 clusters proved to perform the best. With even further investigation, games were extended to 100 for the 65 clusters then compared to the initial trial results, and still 65 clusters performed the best. With these results, 65 clusters was determined to be the most optimal for our data preprocessing needs.

Sentence embeddings were generated from a pre-trained model called "all-mpnet-base-v2" from Microsoft. This model has been trained with over one billion training pairs with a majority coming from Reddit comments, and several other datasets from websites such as Stack Exchange, WikiAnswers, and Yahoo Answers[3]. Red and green card

[3] SBERT model can be found here: https://huggingface.co/sentence-transformers/all-mpnet-base-v2

sentences were encoded into the model where each sentence is placed into a 768-dimensional dense vector space distinctly represented by a 768-dimensional vector.

Some example clusters are shown below in example A: primarily actors and movie characters; and in example B: primarily places and locations. These results were obtained by choosing 20 clusters.

Example A:
Jim Carrey, Jack Nicholson, Keanu Reeves, Mel Gibson, Robert DeNiro, Tom Hanks, Denzel Washington, Bruce Willis, Bill Murray, Nicholas Cage, John Travolta, Adam Sandler, Cary Grant, Sean Connery, Robin Williams, Brad Pitt, Clint Eastwood, Will Smith, Samuel L. Jackson, Steven Spielberg, Clark Gable, Danny DeVito, Humphrey Bogart, Tom Cruise, Anthony Hopkins, Al Pacino, Jackie Chan, Ben Stiller, Woody Allen, Spike Lee, Austin Powers, James Dean, Steve Martin, Meg Ryan, Demi Moore, Meryl Streep, James Bond, Jodie Foster, Julia Roberts, Eddie Murphy, Michelle Pfeiffer, Darth Vader, Katherine Hepburn, Indiana Jones, Captain Kirk, Quentin Tarantino, Michael Jordan

Example B:
Canada, Europe, Mexico, Japan, China, Russia, South Africa, Jamaica, Shopping Mall, Vietnam, Italy, Antarctica, California, My Bedroom, Hawaii, My Neighborhood, The Midwest, Cuba, India, The South, Ireland, My Bathroom, Canadians, Australian Outback, The Ocean, Israel, Redwood Forests, Tibet, Trailer Parks, Dark Alleys, Victorian England, European, The Beach, American, Caves, Casinos, Foreign, Wall Street, East Coast, A Used Car Lot, Japanese, Castles, Homeless Shelters, A Morgue, A School Cafeteria, At My Parent's House, Mars, A Locker Room, Log Cabins, A Cheap Motel, A Crawl Space, A High School Bathroom, A Tree House, The YMCA, Waterbeds, Global, A Haunted House, In A Coma, Swiss Bank Accounts

Large Language Models

A crucial aspect to the success of a reinforcement learning agent is the agents that play amongst them.

By providing informative agents in the reinforcement learning agent's environment, the agent can learn from useful information in the form of judging by the players it plays against. Designing informative agents with the use of Large Language Models allowed for accurate card association and the most optimal solution for judging the agent's plays. With state-of-the-art models such as Llama 3.1 from Meta[4] and Gemma 2 from Google[5], agents playing against reinforcement learning used the power of LLMs and their billions of parameter models to make accurate judging decisions. Additionally, a sentence transformer model agent using SBERT (Reimers, Guryvech 2019) was developed as well since it was already used for the clustering approach. Including the Spacy LLM agent, these 4 agents were used during various parts of testing Agent B.

Large Language Models such as Llama 3.1 and Gemma 2 were run locally with the use of Ollama[6]. Ollama allows for downloading the 4.7GB, 8 billion parameter Llama 3.1 model, 5.5GB, 9 billion parameter Gemma 2 model, and integrating the model's functionality using their Python library. The models can be prompted by giving it a list of the agent's red card and asking which red card is most associated with a given green card. The same goes with judging; given a list of agent's cards played, which card is most associated with a given green card. With the use of the latest technology in AI models, developing agents to play the game was effortless and effective. The SBERT agent used a different approach with sentence transformers.

Given the agent's hand of red cards and a green card, card data was encoded into the model producing sentence embeddings. Similarity scores between each red card and the green card were calculated using Semantic Text Similarity, which calculates cosine similarity between a red card embedding and the green card embedding. The highest similarity score was then used as the agent's play for the round or judge.

Results

The reinforcement learning agent, Agent B, played against Llama 3.1, Gemma 2, and SBERT agents over 400 games. For each game, the sum of its rewards was logged to track how well Agent B played each game (seen in figure 3). Each

[4] Meta's Llama 3.1 model: https://ai.facebook.com/research/llama
[5] Google's Gemma 2 model: https://ai.facebook.com/research/gemma
[6] Ollama: https://ollama.com

round Agent B either received a +1 reward for winning the round and a -0.5 reward for losing the round. With four players, the total score to win the game was 8 rounds won. A running average of the sum of rewards was calculated to see a trend in Agent B's overall performance as it played each game. After a few games, Agent B is seen stabilizing its sum of reward average with no clear indication of an upwards trend.

In a separate sample of 100 games with a trained policy of 400 games, total scores and wins were evaluated to see overall performance in winning the game. The reinforcement learning agent successfully won a total of 12 games out of the 100 games played and won a total of 423 rounds out of 2052 rounds played (seen in figures 1 and 2 respectively).

Discussion

Agent B, although poorly performing relative to the other agents, was able to secure a similar amount of game wins to the SBERT agent, and a comparable number of round wins to the other agents. This may be due to the formulation of Agent B's goal: the agent is rewarded for winning rounds, not necessarily games, so while it may attempt to maximize the expected reward by winning rounds, it does so slowly, causing Agent B to lose games. This could be remedied by changing the formulation of the state space to reflect the goal of winning games, or by making Agent B more shortsighted in terms of rewards by giving it a discount rate of 0. While this may be the case, Agent B demonstrates some level of play, being able to win games against models who are capable of natural language processing.

Further Work

While the Agent B is certainly capable of playing Apples to Apples, its performance seems lackluster (approximately 15 according to the figures). This may be due to the learning parameters not being fine-tuned for this problem, or perhaps the representation of the state space is insufficient; since Agent B may make the same move twice for the same state but receive two different rewards based on the cards the opponents choose, this may make learning unstable.

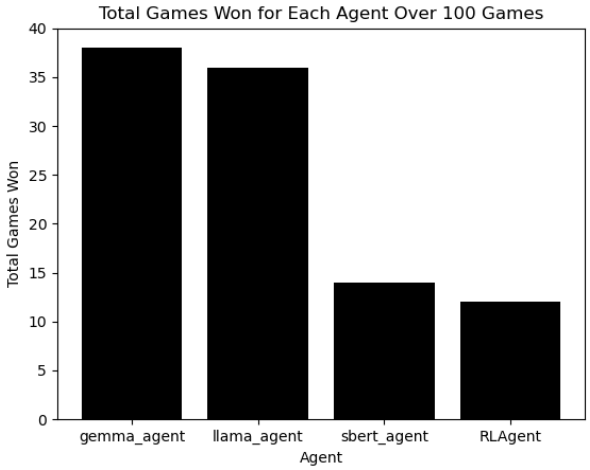


Figure 1 - Total number of games won for each agent over the course of 100 games

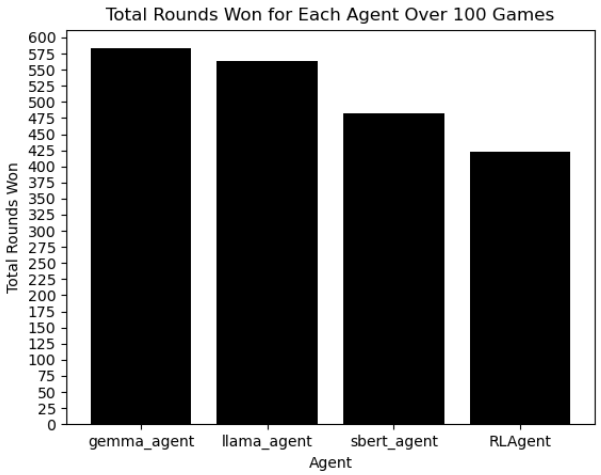


Figure 2 - Total number of game rounds won for each agent over the 100 games depicted in figure 1

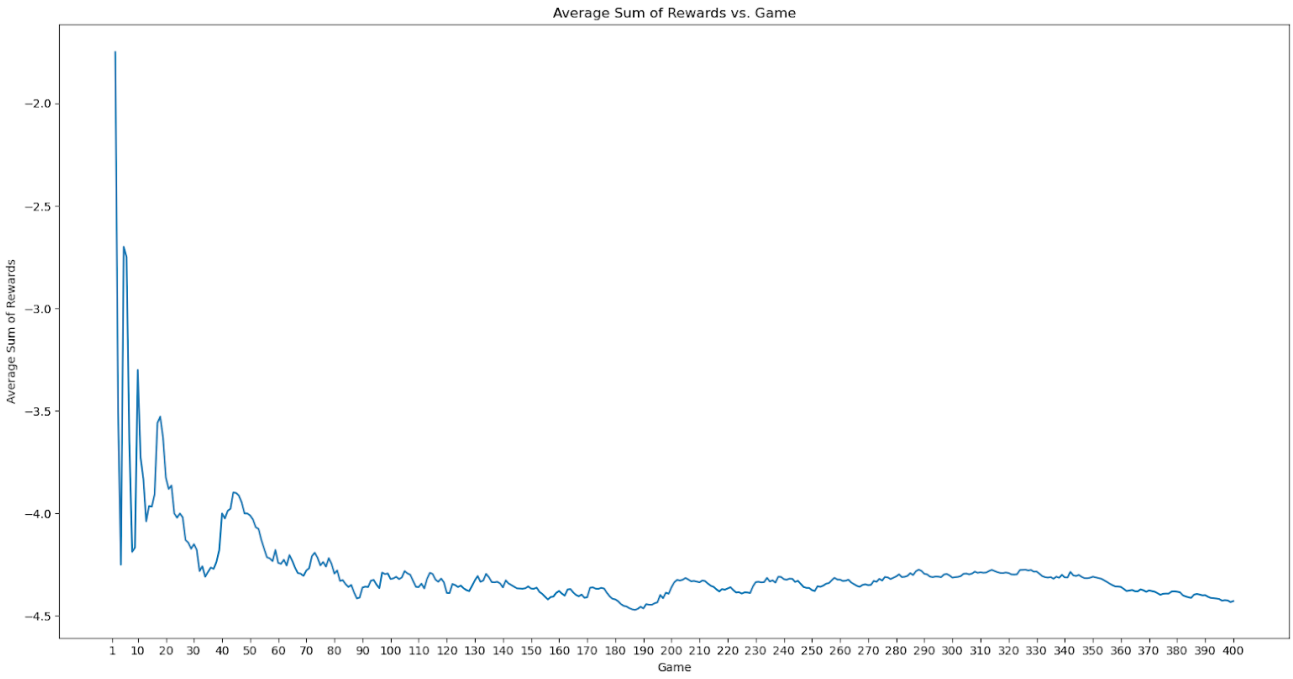


Figure 3 - Average sum of rewards for the reinforcement learning agent over 400 games

A potential alternative state space representation would be to include which cards are still available to be played that are considered a better pick for the green card currently in play. This however would increase the dimensionality and size of the state space significantly, slowing learning even more. Additionally, one would require an objective measure of what a "good" red card for an in play green card is, something that was specifically avoided during the implementation of the reinforcement learning agent.

While Agent B was compared to agents using natural language processing and large language models, no comparison was made between Agent B and human players. This would require more time and work but would provide an objective measure of Agent B's performance.

Part of the appeal of Apples to Apples is the subjectivity involved in the judge's decision on which red card is the winning card. The Apples to Apples handbook even suggests this is a valid way to play, stating that judges may pick the card that is considered the most humorous. Going into the project, the idea to learn different policies for each judge was considered but dropped due to lack of time for implementation. Both agents thus only play the game in the "traditional" way, or at the very least plays best in this context.

A reinforcement learning agent with no penalty for losing was considered as well but was not fully implemented by the project's conclusion.

Conclusion

The development of an artificial intelligence agent capable of rationally playing Apples to Apples was attempted initially through the process of natural language processing and clustering. The initial implementation, although successful, did not show adaptation. The development of Agent B using reinforcement learning aimed to show more growth and adaptation within the game. This technique allowed Agent B to grow towards making contextually informed decisions in an environment that is often socially dominated. Although results were only partially successful, this project shows evidence that there is potential for more definitive results with tweaks to Agent B's learning and cluster sizes.

References

Crespo, J., Wichert, A. 2020. Reinforcement learning applied to games. SN Applied Sciences, 2(824). https://doi.org/10.1007/s42452-020-2560-3.

Fan, J., Wang, Z., Xie, Y., Yang, Z. 2020. A theoretical analysis of deep Q-learning. In: Proceedings of the 2nd Conference on Learning for Dynamics and Control, vol. 120, PMLR, pp. 486-489. https://proceedings.mlr.press/v120/yang20a.html.

Ghory, I. 2004. Reinforcement learning in board games. Department of Computer Science, University of Bristol, Tech. Rep. 105.

Jang, B., Kim, M., Harerimana, G., Kim, J. W. 2019. Q-learning algorithms: A comprehensive classification and applications. IEEE Access, 7, pp. 133653-133667. https://doi.org/10.1109/ACCESS.2019.2941229.

Leslie, D. S., Collins, E. J. 2005. Individual Q-learning in normal form games. SIAM Journal on Control and Optimization, 44(2), pp. 495-514. https://doi.org/10.1137/S0363012903437976.

N. Reimers, Gurevych, I. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 3982-3993. https://arxiv.org/abs/1908.10084.

Osborne, P., Nõmm, H., Freitas, A. 2022. A survey of text games for reinforcement learning informed by natural language. Transactions of the Association for Computational Linguistics, 10, pp. 873-887. https://doi.org/10.1162/tacl_a_00495.

Narasimhan, K., Kulkarni, T., Barzilay, R. 2015. Language understanding for text-based games using deep reinforcement learning. arXiv:1506.08941. https://arxiv.org/abs/1506.08941.

Szita, I. 2012. Reinforcement learning in games. In: Wiering, M., van Otterlo, M. (eds) Reinforcement learning: Adaptation, learning, and optimization, vol. 12. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-27645-3_17.